

```

import pandas as pd
import re
import nltk
import matplotlib.pyplot as plt
import seaborn as sns
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer, WordNetLemmatizer
from nltk.tokenize import word_tokenize, sent_tokenize
from nltk import pos_tag
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report
from collections import Counter
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader
import numpy as np
from torch.nn.utils.rnn import pad_sequence
import zipfile
import os
import warnings
warnings.filterwarnings('ignore') # Suppress warnings for cleaner output

# Optional: spaCy for advanced NLP (POS, NER)
try:
    import spacy # Check if 'en_core_web_sm' is downloaded, if not, skip spaCy features
    try:
        nlp = spacy.load("en_core_web_sm")
        SPACY_AVAILABLE = True
    except OSError:
        print("spaCy model 'en_core_web_sm' not found. Skipping spaCy features. Please download it if needed.")
        SPACY_AVAILABLE = False
        nlp = None
except (ImportError):
    print("spaCy not available. Skipping spaCy features. Install with: pip install spacy")
    SPACY_AVAILABLE = False
    nlp = None

# Download required nltk resources
# Note: In a live environment, these are usually downloaded once.
# In this environment, we download them to ensure they are available.
nltk.download('punkt_tab', quiet=True) # Updated to 'punkt_tab' for newer NLTK versions
nltk.download('stopwords', quiet=True)
nltk.download('wordnet', quiet=True)
nltk.download('omw-1.4', quiet=True)
nltk.download('averaged_perceptron_tagger', quiet=True)

# General POS tagger for NLTK
def process_airline_dataset(csv_path):
    # Handle potential zip file
    file_path = csv_path
    if zipfile.is_zipfile(file_path):
        print("Detected zip file. Extracting CSV...")
        with zipfile.ZipFile(file_path, 'r') as z:
            csv_names = [n for n in z.namelist() if n.endswith('.csv')]
            if not csv_names:
                raise ValueError("No CSV file found in zip.")
            csv_name = csv_names[0]
            with z.open(csv_name) as f:
                df = pd.read_csv(f)
    else:
        if not os.path.exists(file_path):
            raise FileNotFoundError(f"File not found: {file_path}")
        df = pd.read_csv(file_path)

    # --- Data Cleaning and Labeling for BA Reviews ---
    df.columns = df.columns.str.replace('[^A-Za-z0-9_]+', '', regex=True)
    df = df.rename(columns={'ReviewBody': 'review', 'OverallRating': 'rating'})
    df = df[['review', 'rating']].dropna()
    df['rating'] = pd.to_numeric(df['rating'], errors='coerce')
    df = df.dropna(subset=['rating'])
    df['rating'] = df['rating'].astype(int)

```

```

# Convert 1-10 rating to binary sentiment:
# 7-10: Positive (1)
# 1-4: Negative (0)
# Drop 5 and 6 (Neutral/Mixed)
df_positive = df[df['rating'] >= 7].copy()
df_positive['sentiment'] = 1
df_negative = df[df['rating'] <= 4].copy()
df_negative['sentiment'] = 0
df_final = pd.concat([df_positive, df_negative]).reset_index(drop=True)
print(f"Loaded dataset shape: {df.shape}")
print(f"Final dataset for binary classification (ratings 5/6 dropped) shape: {df_final.shape}")
print(f"Sentiment distribution:\n{df_final['sentiment'].value_counts()}")

# Sample texts for basic processing (200), and full set for models (up to 4000)
max_samples = 4000
df_model = df_final.head(max_samples)
texts = df_model['review'].head(200).tolist()
full_texts = df_model['review'].tolist()
full_labels = df_model['sentiment'].tolist()

# Tokenization (NLTK)
tokens_nltk = [word_tokenize(text.lower()) for text in texts[:5]]

# Stemming
stemmer = PorterStemmer()
stems = [[stemmer.stem(word) for word in word_tokenize(text.lower())] for text in texts[:5]]

# Lemmatization (NLTK)
lemmatizer = WordNetLemmatizer()
lemmas_nltk = [[lemmatizer.lemmatize(word) for word in word_tokenize(text.lower())] for text in texts[:5]]

# spaCy tokenization and lemmatization if available
tokens_spacy = None
lemmas_spacy = None
if SPACY_AVAILABLE:
    tokens_spacy = [[token.text for token in nlp(text.lower())] for text in texts[:5]]
    lemmas_spacy = [[token.lemma_ for token in nlp(text.lower())] for text in texts[:5]]

# Stopword removal, regex cleaning, normalization
stop_words = set(stopwords.words("english"))
cleaned_texts = []
for text in full_texts:
    text = str(text).lower() # Ensure string
    text = re.sub(r"[^a-z\s]", "", text) # Keep only letters
    tokens = word_tokenize(text)
    tokens = [word for word in tokens if word not in stop_words and len(word) > 1] # Remove single-char tokens
    cleaned_texts.append(" ".join(tokens))

# Limit cleaned_texts for graphs to first 200
cleaned_texts_sample = cleaned_texts[:200]

# Bag of Words & TF-IDF (small for demo)
vectorizer_bow = CountVecorizer(max_features=20)
bow_features = vectorizer_bow.fit_transform(cleaned_texts_sample).toarray()
bow_vocab = vectorizer_bow.get_feature_names_out()

vectorizer_tfidf = TfidfVecorizer(max_features=20)
tfidf_features = vectorizer_tfidf.fit_transform(cleaned_texts_sample).toarray()
tfidf_vocab = vectorizer_tfidf.get_feature_names_out()

# POS tagging & NER
if SPACY_AVAILABLE:
    pos_tags = [[(token.text, token.pos_) for token in nlp(text)] for text in texts[:5]]
    pos_tags_all = []
    entities_all = []
    for text in texts[:50]:
        doc = nlp(text)
        pos_tags_all.extend([(token.pos_ for token in doc)])
        entities_all.extend([(ent.label_ for ent in doc.ents)])
    entities = [[(ent.text, ent.label_) for ent in nlp(text).ents] for text in texts[:5]]
else:
    pos_tags = []
    for text in texts[:5]:
        tokens = word_tokenize(text)
        pos_tags.append(pos_tag(tokens)[:10]) # First 10
    pos_tags_all = []
    for text in texts[:50]:
        tokens = word_tokenize(text)

```

```

        pos_tags_all.extend([tag for _, tag in pos_tag(tokens)])
    entities_all = []
    entities = [[] for _ in texts[:5]] # Empty

# Sentence segmentation
sentences = [sent_tokenize(text) for text in texts[:5]]

# ---- PRINT OUTPUTS ----
print("\n" + "="*50)
print("## 🚀 NLP Preprocessing Summary (BA Airline Reviews) ##")
print("="*50)
print("🔴 Original Text (Sample):\n", texts[0][:300], "... \n")
print("🔵 Tokenization (NLTK):\n", tokens_nltk[0][:10], "... \n")
if tokens_spacy:
    print("🔵 Tokenization (spaCy):\n", tokens_spacy[0][:10], "... \n")
print("🔵 Stemming:\n", stems[0][:10], "... \n")
print("🔵 Lemmatization (NLTK):\n", lemmas_nltk[0][:10], "... \n")
if lemmas_spacy:
    print("🔵 Lemmatization (spaCy):\n", lemmas_spacy[0][:10], "... \n")
print("🔵 Cleaned & Normalized Text (Sample):\n", cleaned_texts_sample[0][:300], "... \n")
print("🔵 Bag of Words Vocab (Top 20):\n", bow_vocab, "\n")
print("🔵 TF-IDF Vocab (Top 20):\n", tfidf_vocab, "\n")
if SPACY_AVAILABLE:
    print("🔵 POS Tags (spaCy - Sample):\n", pos_tags[0][:5], "... \n")
    print("🔵 Named Entities (spaCy - Sample):\n", entities[0], "\n")
else:
    print("🔵 POS Tags (NLTK - Sample):\n", pos_tags[0], "\n")
print("🔵 Sentence Segmentation (Sample):\n", sentences[0][:2], "... \n")

# ---- 📊 GRAPHS ----
print("\n" + "="*50)
print("## 📊 Visualization of Features ##")
print("="*50)

# 1. Top word frequencies
all_words = " ".join(cleaned_texts_sample).split()
word_freq = Counter(all_words).most_common(20)
words, counts = zip(*word_freq)
plt.figure(figsize=(10,5))
sns.barplot(x=list(counts), y=list(words), palette="viridis")
plt.title("Top 20 Word Frequencies (After Cleaning)")
plt.xlabel("Frequency")
plt.ylabel("Word")
plt.tight_layout()
plt.show()
print("")

# 2. Bag of Words Heatmap
plt.figure(figsize=(10,6))
sns.heatmap(bow_features[:10], annot=True, fmt="d", cmap="Blues", xticklabels=bow_vocab, yticklabels=[f"Doc {i}" for i in range(10)])
plt.title("Bag of Words Feature Representation (first 10 docs)")
plt.xlabel("Vocabulary Terms")
plt.ylabel("Document Index")
plt.tight_layout()
plt.show()
print("")

# 3. TF-IDF Heatmap
plt.figure(figsize=(10,6))
sns.heatmap(tfidf_features[:10], annot=False, cmap="viridis", xticklabels=tfidf_vocab, yticklabels=[f"Doc {i}" for i in range(10)])
plt.title("TF-IDF Feature Representation (first 10 docs)")
plt.xlabel("Vocabulary Terms")
plt.ylabel("Document Index")
plt.tight_layout()
plt.show()
print("")

# 4. POS tag counts
if pos_tags_all:
    pos_freq = Counter(pos_tags_all).most_common(20)
    pos_labels, pos_counts = zip(*pos_freq)
    plt.figure(figsize=(10,5))
    sns.barplot(x=list(pos_labels), y=list(pos_counts), palette="rocket")
    plt.title("Part-of-Speech Tag Distribution (sample reviews)")
    plt.xlabel("POS Tag")
    plt.ylabel("Frequency")
    plt.xticks(rotation=45, ha='right')
```

```

plt.tight_layout()
plt.show()
print("")

# 5. Named Entity frequencies
if entities_all:
    ent_freq = Counter(entities_all).most_common(20)
    ent_labels, ent_counts = zip(*ent_freq)
    plt.figure(figsize=(10,5))
    sns.barplot(x=list(ent_labels), y=list(ent_counts), palette="mako")
    plt.title("Named Entity Types Frequency (sample reviews)")
    plt.xlabel("Entity Type")
    plt.ylabel("Frequency")
    plt.xticks(rotation=45, ha='right')
    plt.tight_layout()
    plt.show()
    print("")
else:
    print("\nNo Named Entities found in sample or spaCy is unavailable.")

# --- 🚀 MODEL TRAINING ---
# === TF-IDF Word Vectors for Models ===
vectorizer_tfidf_model = TfidfVectorizer(max_features=5000)
tfidf_features_model = vectorizer_tfidf_model.fit_transform(cleaned_texts).toarray()
tfidf_vocab_model = vectorizer_tfidf_model.get_feature_names_out()[:20]
print("\n" + "="*50)
print("### 🚀 Classical ML Model: Logistic Regression ###")
print("="*50)
print("TF-IDF Model Vocab sample:", tfidf_vocab_model)
print("TF-IDF features shape:", tfidf_features_model.shape)

# Split data
X_train, X_test, y_train, y_test = train_test_split(tfidf_features_model, full_labels, test_size=0.2, random_state=42, stratify=y_train)

# === Logistic Regression with TF-IDF ===
lr_model = LogisticRegression(max_iter=1000)
lr_model.fit(X_train, y_train)
lr_pred = lr_model.predict(X_test)
lr_acc = accuracy_score(y_test, lr_pred)
print("\n=== Logistic Regression Results ===")
print(f"Accuracy: {lr_acc:.4f}")
print("\nClassification Report:\n", classification_report(y_test, lr_pred, target_names=['Negative (0)', 'Positive (1)']))
print("---")

# === Preparing Sequences for LSTM ===
# Build vocab for sequences
all_tokens = [word_tokenize(text) for text in cleaned_texts]
vocab = {'<PAD>': 0, '<UNK>': 1}
for tokens in all_tokens:
    for token in tokens:
        if token not in vocab:
            vocab[token] = len(vocab)
vocab_size = len(vocab)

# Convert to indices, pad to 200
def text_to_indices(texts, max_len=200):
    indices = []
    for text in texts:
        toks = word_tokenize(text)
        idx = [vocab.get(t, vocab['<UNK>']) for t in toks][:max_len]
        indices.append(torch.tensor(idx, dtype=torch.long))
    return pad_sequence(indices, batch_first=True, padding_value=0)

# Calculate split point (80% for train, 20% for test)
split_point = int(len(cleaned_texts) * 0.8)
train_sequences = text_to_indices(cleaned_texts[:split_point])
train_labels = torch.tensor(full_labels[:split_point], dtype=torch.long)
test_sequences = text_to_indices(cleaned_texts[split_point:])
test_labels = torch.tensor(full_labels[split_point:], dtype=torch.long)

class TextDataset(Dataset):
    def __init__(self, sequences, labels):
        self.sequences = sequences
        self.labels = labels

    def __len__(self):
        return len(self.sequences)

```

```

def __getitem__(self, idx):
    return self.sequences[idx], self.labels[idx]

train_dataset = TextDataset(train_sequences, train_labels)
test_dataset = TextDataset(test_sequences, test_labels)
train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=32)

print("\n" + "="*50)
print("## 🌸 Deep Learning Models: LSTM & BiLSTM ##")
print("="*50)
print(f"Sequence Training Size: {len(train_sequences)}, Testing Size: {len(test_sequences)}")
print(f"Vocabulary Size: {vocab_size}")

# === LSTM Model ===
class LSTMModel(nn.Module):
    def __init__(self, vocab_size, embed_dim=100, hidden_dim=128, num_layers=1):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim, padding_idx=0)
        self.lstm = nn.LSTM(embed_dim, hidden_dim, num_layers, batch_first=True, dropout=0.5 if num_layers > 1 else 0)
        self.fc = nn.Linear(hidden_dim, 2)
        self.dropout = nn.Dropout(0.5)

    def forward(self, x):
        embedded = self.embedding(x)
        _, (hidden, _) = self.lstm(embedded)
        hidden = self.dropout(hidden[-1])
        return self.fc(hidden)

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
lstm_model = LSTMModel(vocab_size).to(device)
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(lstm_model.parameters(), lr=0.001)

print("\n=== Training LSTM (5 Epochs) ===")
lstm_model.train()
for epoch in range(5):
    total_loss = 0
    for seq, lbl in train_loader:
        seq, lbl = seq.to(device), lbl.to(device)
        optimizer.zero_grad()
        out = lstm_model(seq)
        loss = criterion(out, lbl)
        loss.backward()
        optimizer.step()
        total_loss += loss.item()
    print(f"LSTM Epoch {epoch+1}, Avg Loss: {total_loss / len(train_loader):.4f}")

# Evaluate LSTM
lstm_model.eval()
preds, trues = [], []
with torch.no_grad():
    for seq, lbl in test_loader:
        seq = seq.to(device)
        out = lstm_model(seq)
        pred = torch.argmax(out, dim=1)
        preds.extend(pred.cpu().numpy())
        trues.extend(lbl.numpy())
lstm_acc = accuracy_score(trues, preds)
print(f"\nLSTM Accuracy: {lstm_acc:.4f}")
print("\nClassification Report:\n", classification_report(trues, preds, target_names=['Negative (0)', 'Positive (1)']))
print("---")

# === BiLSTM Model ===
class BiLSTMModel(nn.Module):
    def __init__(self, vocab_size, embed_dim=100, hidden_dim=128, num_layers=1):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim, padding_idx=0)
        self.bilstm = nn.LSTM(embed_dim, hidden_dim, num_layers, batch_first=True, bidirectional=True, dropout=0.5 if num_la
        self.fc = nn.Linear(hidden_dim * 2, 2) # *2 for bidirectional
        self.dropout = nn.Dropout(0.5)

    def forward(self, x):
        embedded = self.embedding(x)
        _, (hidden, _) = self.bilstm(embedded)
        # Concatenate the final forward (hidden[-2]) and backward (hidden[-1]) hidden states
        hidden = self.dropout(torch.cat((hidden[-2, :, :], hidden[-1, :, :]), dim=1))

```

```

        return self.fc(hidden)

bilstm_model = BiLSTMModel(vocab_size).to(device)
optimizer = optim.Adam(bilstm_model.parameters(), lr=0.001)

print("\n=== Training BiLSTM (10 Epochs) ===")
bilstm_model.train()
for epoch in range(10):
    total_loss = 0
    for seq, lbl in train_loader:
        seq, lbl = seq.to(device), lbl.to(device)
        optimizer.zero_grad()
        out = bilstm_model(seq)
        loss = criterion(out, lbl)
        loss.backward()
        optimizer.step()
        total_loss += loss.item()
    print(f"BiLSTM Epoch {epoch+1}, Avg Loss: {total_loss / len(train_loader):.4f}")

# Evaluate BiLSTM
bilstm_model.eval()
preds, trues = [], []
with torch.no_grad():
    for seq, lbl in test_loader:
        seq = seq.to(device)
        out = bilstm_model(seq)
        pred = torch.argmax(out, dim=1)
        preds.extend(pred.cpu().numpy())
        trues.extend(lbl.numpy())
bilstm_acc = accuracy_score(trues, preds)
print(f"\nBiLSTM Accuracy: {bilstm_acc:.4f}")
print("\nClassification Report:\n", classification_report(trues, preds, target_names=['Negative (0)', 'Positive (1)']))
print("---")

print("\nNote: BERT model requires the 'transformers' library, which is not available in this environment.")

# ---- Run the pipeline with the BA Airline Reviews dataset ----
# The correct file path for the uploaded zip content is used here.
process_airline_dataset("/content/BA_AirlineReviews.csv.zip")

```


Detected zip file. Extracting CSV...

Loaded dataset shape: (3696, 2)

Final dataset for binary classification (ratings 5/6 dropped) shape: (3272, 3)

Sentiment distribution:

```
sentiment
0    1949
1    1323
```

Name: count, dtype: int64

🚀 NLP Preprocessing Summary (BA Airline Reviews) ##
#####

📌 Original Text (Sample):

Boarded on time, but it took ages to get to the runway due to congestion. Flight was smooth and quick. Snack and drinks were go

◆ Tokenization (NLTK):

['boarded', 'on', 'time', ',', 'but', 'it', 'took', 'ages', 'to', 'get'] ...

◆ Tokenization (spaCy):

['boarded', 'on', 'time', ',', 'but', 'it', 'took', 'ages', 'to', 'get'] ...

◆ Stemming:

['board', 'on', 'time', ',', 'but', 'it', 'took', 'age', 'to', 'get'] ...

◆ Lemmatization (NLTK):

['boarded', 'on', 'time', ',', 'but', 'it', 'took', 'age', 'to', 'get'] ...

◆ Lemmatization (spaCy):

['board', 'on', 'time', ',', 'but', 'it', 'take', 'age', 'to', 'get'] ...

◆ Cleaned & Normalized Text (Sample):

boarded time took ages get runway due congestion flight smooth quick snack drinks good short flight landed ten minutes late one

◆ Bag of Words Vocab (Top 20):

['ba' 'boarding' 'cabin' 'class' 'club' 'comfortable' 'crew' 'first'
'flight' 'food' 'friendly' 'good' 'great' 'heathrow' 'london' 'seat'
'seats' 'service' 'time' 'well']

◆ TF-IDF Vocab (Top 20):

['ba' 'boarding' 'cabin' 'class' 'club' 'comfortable' 'crew' 'first'
'flight' 'food' 'friendly' 'good' 'great' 'heathrow' 'london' 'seat'
'seats' 'service' 'time' 'well']

◆ POS Tags (spaCy - Sample):

[('Boarded', 'VERB'), ('on', 'ADP'), ('time', 'NOUN'), (',', 'PUNCT'), ('but', 'CCONJ')] ...

◆ Named Entities (spaCy - Sample):

[('only about ten minutes', 'TIME'), ('One', 'CARDINAL'), ('three', 'CARDINAL'), ('London', 'GPE'), ('the next morning', 'TIME')]

◆ Sentence Segmentation (Sample):

['Boarded on time, but it took ages to get to the runway due to congestion.', 'Flight was smooth and quick.'] ...

📊 Visualization of Features ##
#####



